# Agent 评估体系：架构、流程与落地实践

标准录音 13录音 · 2026/10/23 15:34 - 16:15 · 整理于 2026/10/23 15:52

## 一、会议议程与关键信息

- 不同角色对评估体系的需求差异分析；
- 评估架构三大核心模块（评估资产、评估流水线、评估配置）；
- 评估流程与多维度评估标准（结果评估、意图评估、轨迹评估）；
- 数据集来源与建设方式（自写用例、生产流量回流、异常数据筛选）；
- 评估器类型（代码评估器、AI 评估器）与人工校验机制；
- 数据清洗与轨迹提取流程；
- 通用能力与业务定制化需求的矛盾及解决方案；
- 评估引擎架构设计（工作流与后端服务分离）及优化方向；
- 评估指标与业务反馈机制；
- 平台定位与未来合作模式展望。

## 二、不同角色对评估体系的需求差异

| 角色 | 需求特点 |
| --- | --- |
| 产品视角 | 关注整体效果——用户是否收到倡议（反馈）、满意度等；产运也关心成本是否上升。希望看到全局视角的评估，以及改完之后的版本和上个版本的区别：如果钱花得更多、成本更大、用户满意度又下降，那说明这个 agent 做得不好。 |
| 轻量 agent 开发者 | 需求简单：有没有现成的平台帮忙简单跑一下，看任务到底完成怎么样、回答得好不好，有没有一些评估结果可用来改进——即"开箱即用"。 |
| 深度 agent 开发者（如法院托底、搜推） | 做的评估非常符合标准，且标准不只是简单的任务完成率：先做初筛统计，再做倾向打分，小于阈值再去深度打分，也可能有不同评估机制并发评估后综合打分。不只看结果——例如黄金测试题有标准答案，看 agent 有没有调用到该调的图，调用成功不代表 OK，一定要看各种参数是否满足整个调用链的关系、是否按需求来。例如下单购物场景，结果判断不是简单能搞定的，需要做代码级别的嵌入。 |

**需求演进的普遍规律**：写过 agent 的人基本都经历过——先把 agent 写出来，找平台帮忙简单评一下，发现结果好像挺好，但评判标准跟自己的不一样；于是开始自己制定标准（"任务完成"对不同 agent 是不一样的）；最后形成一个属于业务自己的评估。

以可观测 agent 为例：如果问一些跟可观测无关的问题，agent 会拒绝；用空数据集去问，它也基本是拒绝性的回答。评估系统会看工具调用——看有没有给出家长建议、参数有没有传对；这个问题答案经过意图识别之后跑一下，看是不是走到意图识别这个模块；线上数据虽然采样了，但采样之后先用小模型做初筛，看到效果不好时再用较大的模型去评估，评完之后的结果再交给人工去校验。

## 三、评估架构三大核心模块

整个评估架构设计成三个大模块：

1. **评估资产**：是评判黄金数据或回归数据非常重要的数据（数字资产）；
2. **评估流水线**：在业务里定义评估流程怎么串起来——这个节点接收什么数据、从哪个上游接数据，都是有讲究的，也是一套非常有用的评估经验（资产）；
3. **评估配置**：即评估器，分类包括代码评估器、AI 评估器等。

**评估器类比高考阅卷**：谁来评——不同大模型相当于写作文，如果所有老师评的分都差不多就没问题；如果有的作文有人给高分、有人给低分，评的结果差异巨大就需要人工重新介入。按什么标准——相当于语文和数学用不同标准去评，可以在不同维度上评。评哪一层——相当于考填空题、选择题还是简答题。真正做评估时是多个标准、多个模块一起协作的：主观题用大模型来评，但主观到底凭什么也要有依据。

### 四个评估维度

1. **结果评估**：输入输出比较明确的 agent，回答好不好很明显；
2. **意图评估**：意图识别准不准也是一个关键评价标准；
3. **轨迹评估**：较困难——例如想验证一个 skill 到底有没有用：skill 的 md 文件有没有被加载？加载完后如果 skill 里有些 SOP，后面有没有相应去调用某个工具或接口，即关键节点的执行轨迹，这是更深一层的评估；
4. **评估的准确性**：有没有统一的评分标准——真实评估体系里不是单单一个 LLM 评估，是四个维度协同。

## 四、评估流程与多维度评估标准

结果评估比较简单，输入输出对得上即可；刚开始大家不会做很深，主要看回答，随着深入之后每个节点可能都要统计耗时。以可观测为例：用户说"我要查最近七天的　CPU"，要看工具调的参数对不对；意图准不准是另一个维度；轨迹评估则深入到第二步、第三步。

**链路经验本身也是评估标准**：例如发现指标异常后，先在链路/日志里看到了　CCQ（监控指标），要继续往下看，看上下游依赖关系——主要依赖了数据库，数据库里如果不是预期的 A 到 B，是不是要深入到 B 里面再看——这其实也是轨迹的评估标准，相当于要写一套东西去评估排查的流程，是一个多因素的方法。

**业务标准**：把整个过程拆成不同的路径。一开始大家比较简单，把数据直接扔给大模型说"我要评估什么什么"，但这不是非常准确的方式。输出数据量大之后，大模型有时候偏向于"论证得好像很有道理"，偏向打高分，实际上可能并不正确、不符合预期。所以业务评估时要把整个过程都变成**可量化评估的方式**，如果没有就补齐。

## 五、数据集来源与建设方式

1. **自写用例**：冷启动上线时自己写的数据肯定不多，脑暴一下可能就 20、30 条，像写测试用例一样直接写到数据集里——但自己写的东西永远是有限的；
2. **生产流量回流**：生产数据量大，需要做粗筛+精筛——量多时可以先采样，采样后经过粗的统计，再经过精的评估，认为回答效果不好的放进问题池；之后要不要直接进数据集，还是需要人工把关；
3. **异常数据筛选**：从链路上看异常——整个 agent 回复时长过长、某一步调用耗时过长、或者里面直接报错了，这些都会进入业务问题数据的基础。

这些最终可能都要人工合并到一个叫**黄金数据集（golden dataset）**的数据资产中。**数据集是现在评估里的重中之重**，来源主要是：自己去想、生产流量直接回流、异常数据直接筛选。

## 六、评估器类型与人工校验机制

- **规则/代码评估器**：写代码是一种；还有一种不用写代码但本质也属于代码评估——结果是固定的，一个问题问进去分类结果就是固定的，只要把结论对出来对比一下就好；
- **AI 评估器（LM 评估）**：偏主观题会采用大模型评估，可能需要不同模型——不同模型回答结果有时不一样，希望有多个 LLM，最后交叉验证；
- **人工校验**：大模型毕竟是大模型，需要一个调教过程。开始时一定需要人工介入，看评判标准到底有什么影响；随着模型不断升级、编排不断发展，评估结果会越来越好。

## 七、数据清洗与轨迹提取流程

高级模式就是**流水线**。数据来源一个是线上数据，还有 session 数据——session 就是一个对话里的数据，用户每问一个问题相当于一个节点，整个会话是一串，每个节点都要做判断。

数据进来后，原始数据要做转化，即**数据清洗**：

- **意义筛选**：有些数据没有评估意义——例如有的业务说客户做完之后推荐数量大于等于 3 的才评估，只有一两个的就不评；
- **轨迹提取**：对话很长，把与本次评估模块相关的轨迹提取出来；
- **格式转换**：转换成评估器共用的输入格式。例如评估器的输入是某一次的工具调用——想评估某次调用，接收的参数对不对，按统一格式喂给评估器。

## 八、通用能力与业务定制化需求的矛盾及解决方案

通用能力在真实使用中发现的矛盾：平台不可能让业务啥都不干——业务啥都不干时平台只能提供最简单的能力让它接进来，比如通用 agent 有一些粗略的评估能力（意图准不准等），甚至可能评不清楚；真正业务评估时一定需要**自定义内容**。而平台方又不可能每个业务单独搞一套。

**解决方案**：

- 提供 **skill** 的方式去定制，这是一种；
- 有些业务觉得还是不能满足，需要更深入的设计，甚至自己写 agent、自己搞一套评估，平台提供一些服务让它知道怎么接入；
- 通用能力最重要的作用是**让用户先用起来**——很多时候用户连怎么用都不知道，重要的是养成使用的习惯；
- 真正跟业务合作时，一定需要跟业务相互定制，用定制的方式才能落地。

**沙箱兼容性教训**：业务写的评估代码一开始用的是平台的沙箱，里面有很多用法不支持——不支持写文件、不支持业务习惯的写法，业务写进去之后跑不起来、兼容不了，还不如自己搞一个。所以通用的能力如果不能真正满足用户需求，纯粹只是建立信任的起点。

## 九、评估引擎架构设计及优化方向

### 三层架构

- **工作流层**：把整个资产串起来——每个节点定义好输入输出，做数据的转化和提取；评估请求在架构里怎么编排、怎么管理；运行之后做实验对比（要上线的 agent 跟之前准备上线的、跟线上 agent 对比）；
- **数据层**：把数据联合起来、做对比、交给下一个环节；
- **引擎层**：具体怎么执行。

### 离线评估与在线评估

- **离线**：数据是离线的。最常用的方式是拿已有的数据集先跑一遍；没有输入输出答案、或没时间做的用户，可以拿今天准备上线的东西跟线上做对比，看看有没有明显差异——这是时间有限情况下比较"偷懒"的方式；
- **在线**：线上数据直接进渠道。在线统计好的数据本质上就是离线的数据集来源，可以配置；数据量大的情况下做采样、按模块/标签统计，作为在线技术评估服务。

### 执行控制

执行机制很简单：任务派发由平台负责执行，公司一般都有 QPS 限制，每分钟要控制量——比如今天要评估 10 万条数据，就按配额控制；**并发**是在同一个时间窗口里最多同时跑多少，像漏斗一样；跑到一半能不能断点续跑也是要处理的；最终拿到结果扔给业务，业务团队有专门做标注的人员去做后续处理——这就是评估引擎。

### 优化方向：从工作流到服务代码

**工作流与后端服务分离的好处**：工作流单独编排，但具体执行不把代码写在工作流里——数据准备虽然走工作流，但本质上是调用后端接口拉数据，全由后端做。这样所有模块可复用（评估器更新后所有工作流立刻生效）；后端评估服务更新后所有工作流自动生效，维护容易；而且公司有自己的工作流引擎，后面要迁移的话成本非常低。

**存在的问题**：真实线上评估里数据准备量很大，链接、图片有可能都要拿下来；如果数据准备→评估→结果处理都走服务调用，每次工作流节点间跨网络调用一定有开销。之前评估 agent 时同一时间并发几百个任务、逻辑不复杂、只是跑一下的模式，对资源浪费很大，很大程度都耗在网络开销上。

**正在做的改造**：把这套东西按配置**生成代码**，直接打包成服务去执行，不走工作流逐节点调用；同时要解决出口资源隔离——打成独立服务后，按业务独立申请资源，不像现在大家共用资源。这样像 K8S 一样能看到这个服务的运行情况。把模块化设计变成一套服务代码，完全做到资源隔离。目的：业务并发速度要求非常高、batch 方式对性能资源消耗很大；另外独立服务没有沙箱"不让写文件"之类的限制，用户可以把代码直接放进去，不用像现在被吐槽"辛辛苦苦写的评估代码放到沙箱上适配不了"，业务的接入体验和效率都会更好。沙箱运行时则自己申请资源、完全隔离。

## 十、评估指标与业务反馈机制

- **上线后效果指标由业务自定义**：例如某业务两个组件上去之后，安全分从 3.98 跌到 3.9——从平台角度可能是波动，但业务认为绝大多数场景都应该是满分，所以 3.98 到 3.9 就是问题。指标标准由业务自己定，不是平台迭代后统一分布就行；
- **一致性**：同一个问题回答的结果在多次评分里差距很大的话，也说明 agent 有问题——大模型是概率性的不确定，但做优化的本质就是让它尽可能确定；agent 上线要"第一次就过"，然后灰度慢慢放量，希望尽量提高一次通过率；
- **CI 流程指标**：评估进入 CI 流程后，上线之前有没有不达标项是一个业务交付指标；
- **结果反馈闭环**：数据给出去之后，分数有问题的通过统计定位——这个问题到底是谁的问题：是 agent（A）的问题还是评估器的问题？本来评得挺好的，改了 agent 之后整体分数直接变化，也可能是评估器本身需要完善——评估器也需要被评估。从在线数据不断完善，真实情况下用户数据也要看得明白。

**平台与业务分工**：数据沉淀在平台侧（所有用户用平台数据很方便，否则数据就散了），业务自己去定义展示；接入之后要测试，测完之后人工判断到底是谁的问题——是 agent 的问题还是评估器的问题——然后不断地维护。报表反映的是资产的变化：过了一年，当时的评估资产、数据集、评估器可能完全会有大的变化，要不断维护；如果发现效果越来越差，要回看之前的数据集和评估器。自建平台的核心就是这个数据结果怎么用。

**下一步**：打通平台——不同业务都有自己的平台，互相合作，千人千面的需求靠合作解决，这是下一个真实迭代的重点。

## 十一、共识与决议

- 评估体系需要满足不同角色（产品、轻量开发者、深度开发者）的差异化需求，平台提供通用能力，业务按需定制；
- 数据集是评估体系的重中之重，来源包括自写用例、生产流量回流、异常数据筛选，需要人工合并到黄金数据；
- 评估器需要多种类型（代码评估器、AI 评估器）协作，且需要人工校验和调教过程；
- 评估引擎采用工作流与后端服务分离的架构，便于复用和更新，并正在向服务化、资源隔离方向演进；
- 评估指标需要业务自定义，且需要持续维护和优化评估资产。

### 关键决议

1. 评估架构设计为三大核心模块：评估资产、评估流水线、评估配置；
2. 评估流程包含数据清洗、轨迹提取、粗筛精筛、人工校验等环节；
3. 评估引擎采用工作流与后端服务分离的架构，并正在推进模块化设计向服务代码转型，实现资源隔离；
4. 平台提供通用能力，业务按需定制，通过 skill 等方式满足业务自定义需求。

### 行动项

1. 将评估配置通过转代码方式输入到平台，实现服务化部署和资源隔离；
2. 将模块化设计变成一套服务代码，完全做到资源隔离，解决 batch 方式对性能资源消耗大的问题；
3. 打通平台，与不同业务自有平台互相合作，推进下一个真实迭代。

由 NEXT AI 生成，内容仅供参考